PISAMA

Agent Forensics Platform

Find out why your AI agent failed. Fix it automatically.

pisama.ai

The Problem

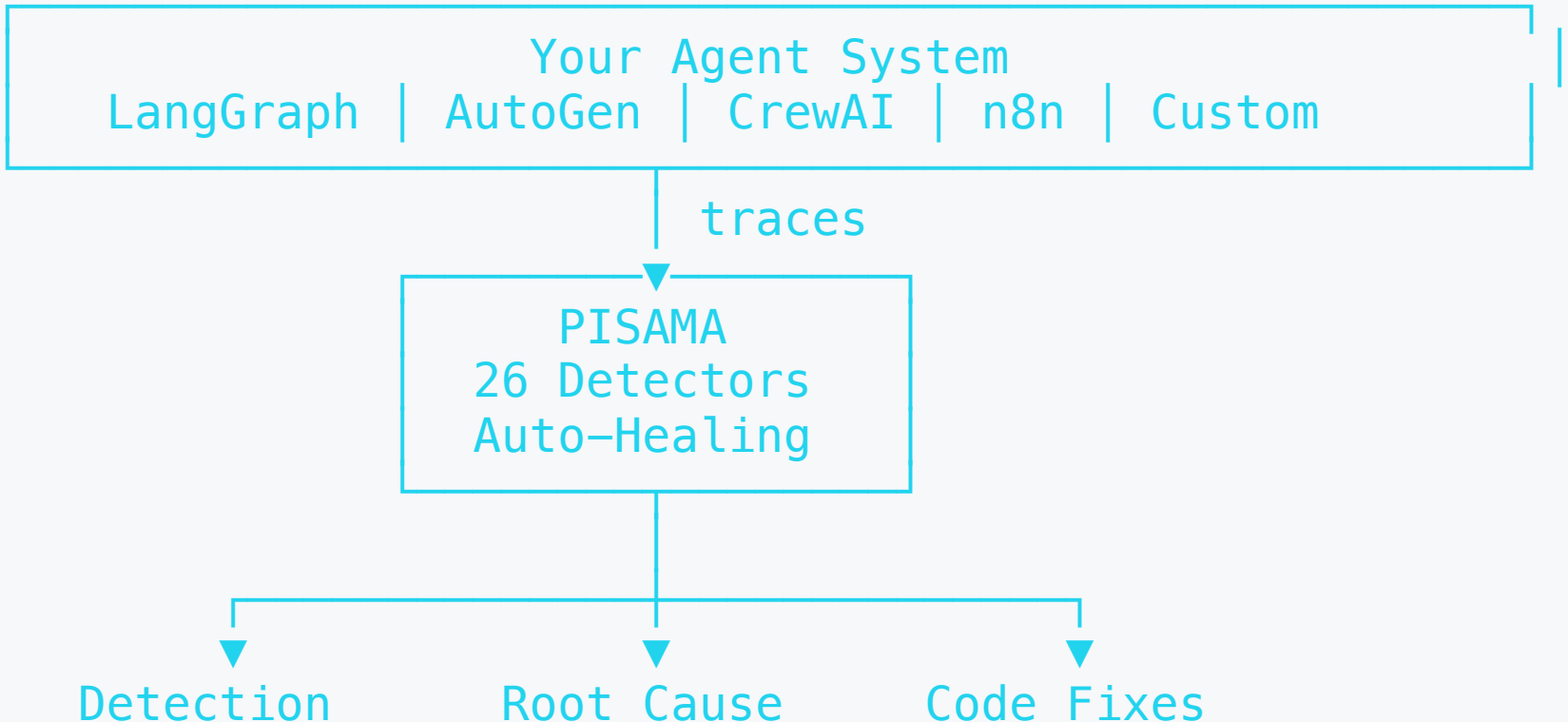
AI Agents Fail in Unpredictable Ways

- **Infinite loops** - Agents repeat the same actions endlessly
- **State corruption** - Data becomes invalid mid-execution
- **Persona drift** - Agents forget their role and instructions
- **Coordination deadlocks** - Multi-agent systems get stuck
- **Goal abandonment** - Tasks silently fail to complete

Current debugging: Manual log reading for hours

The Solution

PISAMA: Automated Agent Diagnostics



What We've Built

Production-Ready Platform

Component	Status	Highlights
Detection Engine	Live	26 failure detectors
Self-Healing	Live	AI-powered fix suggestions
Dashboard	Live	11 views, real-time monitoring
SDK	Live	Python, 4 framework integrations
CLI	Live	CI/CD ready, MCP server
API	Live	REST, multi-tenant
Demo	Live	4 interactive scenarios

Detection Engine

26 Failure Mode Detectors

Behavioral Patterns

- **Loop Detection** - Structural, hash, and semantic analysis
- **State Corruption** - Domain validation, velocity anomalies
- **Persona Drift** - Role consistency scoring (5 role types)
- **Coordination Failures** - Circular delegation, ignored messages

Safety & Security

- **Hallucination Detection** - Grounding scores, citation verification
- **Prompt Injection** - 35+ attack patterns, 13 jailbreak signatures

Detection Engine (continued)

Resource & Performance

- **Context Overflow** - Token warnings at 70%, 85%, 95%
- **Cost Analysis** - 25+ LLM models with 2025 pricing
- **Latency Tracking** - Per-span millisecond precision

Task & Workflow

- **Task Derailment** - Off-topic deviation detection
- **Specification Mismatch** - Output vs requirements comparison
- **Goal Abandonment** - Incomplete task chain detection

Detection Accuracy

MAST 14-Mode Testing Framework

Category	Failure Modes	Accuracy
System Design	Spec mismatch, decomposition, loops, tools, workflow	92%
Inter-Agent	Derailment, context, withholding, coordination, communication	89%
Verification	Corruption, persona drift, quality gates, completion	94%

Detection Methods

- Structural pattern matching
- Semantic similarity (pgvector embeddings)
- LLM-as-Judge with tier escalation

Self-Healing Engine

AI-Powered Fix Suggestions

```
# Detection: Infinite loop between Agent1 and Agent2
# Confidence: 94%

# Suggested Fix: Add circuit breaker
class CircuitBreaker:
    def __init__(self, max_failures=3, reset_timeout=60):
        self.failures = 0
        self.max_failures = max_failures

    def call(self, func, *args):
        if self.failures >= self.max_failures:
            raise CircuitOpenError("Too many failures")
        try:
            return func(*args)
```

Fix Types

Automated Remediation Options

Fix Type	Use Case
Retry Limit	Add configurable retry counters
Exponential Backoff	Progressive wait times
Circuit Breaker	Stop after N failures
State Validation	Add validation wrappers
Schema Enforcement	Enforce data contracts
Prompt Reinforcement	3 levels: light, moderate, aggressive
Role Boundary	Enforce agent responsibilities
Timeout Addition	Prevent infinite waits

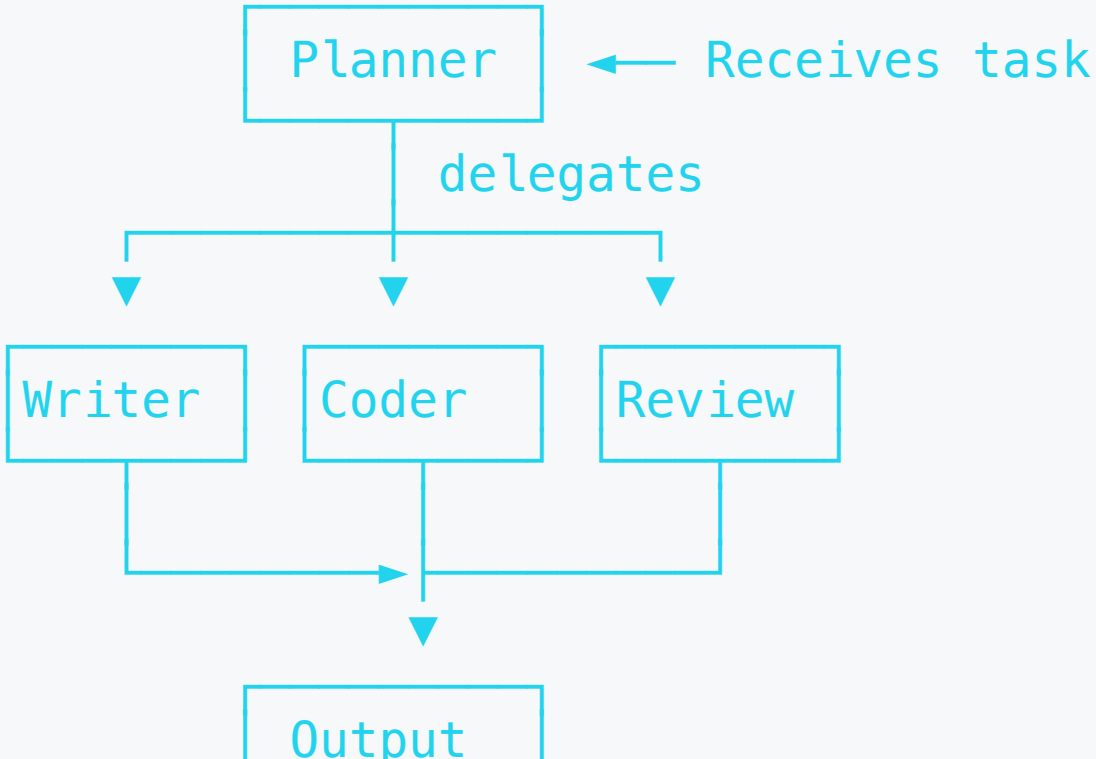
Dashboard

11 Interactive Views

View	Purpose
Dashboard	High-level metrics, recent detections
Agent Orchestration	Network graph of agent communication
Agent Details	Deep-dive into individual agents
Traces	Execution history with filtering
Detections	All failures by type/severity
Agent Forensics	Paste-and-analyze debugging
Testing	14-mode accuracy metrics
Chaos Engineering	Failure injection testing

Agent Orchestration View

Real-Time Multi-Agent Visualization



Agent Forensics

Root Cause Analysis in Seconds

Before PISAMA

> *"The agent failed after 2 hours. I spent 4 hours reading logs and still don't know why."*

With PISAMA

1. Paste your trace (LangSmith, OpenTelemetry, or raw JSON)
2. Click "Diagnose"
3. See

Interactive Demo

4 Failure Scenarios

Scenario	What Happens
Healthy Workflow	Normal execution, all agents complete
Infinite Loop	Agents 1-2-3 repeat endlessly, detected at 94% confidence
State Corruption	Data degrades mid-execution, validation fails
Coordination Deadlock	Agents waiting on each other indefinitely

Live at pisama.ai/demo

- Start/pause simulation
- Watch real-time detection alerts

Python SDK

3-Line Integration

```python
from mao_testing import MAOTracer

tracer = MAOTracer(api_key="your-key")

with tracer.session("my-workflow") as session:
    # Your agent code here
    result = run_my_agents()
    session.snapshot("final_output", result)
```

Features

- OpenTelemetry-based tracing

Framework Integrations

Works With Your Stack

Framework	Integration Level
LangGraph	Full - decorator-based node tracing
AutoGen	Full - agent + conversation tracing
CrewAI	Full - crew, task, and agent tracing
n8n	Full - async polling + webhooks
LangChain	Planned
OpenAI Assistants	Planned
AWS Bedrock Agents	Planned

LangGraph Example

Automatic Node Instrumentation

```python
from mao_testing.integrations import LangGraphTracer
from langgraph.graph import StateGraph

tracer = LangGraphTracer(api_key="your-key")

# Decorate individual nodes
@tracer.trace_node()
def my_agent_node(state):
    return {"messages": [...]}

# Or instrument entire graph
graph = StateGraph(AgentState)
graph.add_node("agent", my_agent_node)
instrumented = tracer.instrument_graph(graph)
```

CLI & DevOps

Production-Ready Tooling

```
# Analyze a specific trace
mao debug trace-abc123

# Get fix suggestions
mao fix detection-xyz789 --apply

# Watch for new detections in real-time
mao watch --severity high

# CI/CD integration with exit codes
mao ci --threshold 0.95
```

Features

MCP Server

Claude Code Integration

```
{
  "mcpServers": {
    "pisama": {
      "command": "mao",
      "args": ["mcp", "serve"]
    }
  }
}
```

Available Tools

- `mao_analyze_trace` - Analyze specific traces

API

RESTful Multi-Tenant Architecture

Core Endpoints

```
POST /traces/ingest           - Accept OTEL traces (async)
GET  /detections              - List with filtering
POST /diagnose/why-failed     - Root cause analysis
POST /diagnose/quick-check    - Fast assessment
GET  /detections/{id}/fixes   - Fix suggestions
POST /chaos/sessions          - Chaos testing
```

Enterprise Features

- Clerk authentication (Google, GitHub, Email SSO)

Technical Architecture

Production Infrastructure

```
┌──────────────────────────────────────────────────────────┐
│                         Frontend                         │
│          Next.js 16 │ React 18 │ TailwindCSS             │
└──────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────┐
│                         Backend                          │
│    FastAPI │ SQLAlchemy │ PostgreSQL │ pgvector          │
├──────────────────┬──────────────────┬──────────────────┤
│ Detection        │ Ingestion        │ Self-Healing     │
│ Engine           │ Pipeline         │ Engine           │
│ (26 algos)       │ (4 formats)      │ (9 fix types)    │
└──────────────────┴──────────────────┴──────────────────┘
```

Trace Ingestion

Multi-Format Support

Format	Features
LangSmith	JSONL/JSON, run types, session mapping
OpenTelemetry	OTEL + OTLP, hierarchical spans
n8n	Workflow logs, node execution
Raw JSON	Custom field mapping
Universal	Framework-agnostic abstraction

Pipeline Features

- Async buffer with backpressure control

Database Design

Optimized for Agent Traces

```
-- Core tables
Tenant (id, name, clerk_org_id, settings JSONB)
Trace  (id, tenant_id, session_id, framework, status)
State  (id, trace_id, sequence_num, embedding VECTOR(1024))

-- Detection storage
Detection (id, trace_id, type, confidence, method, details JSONB)

-- Indexes for fast queries
CREATE INDEX idx_detection_type ON Detection(tenant_id, detection_type);
CREATE INDEX idx_trace_time ON Trace(tenant_id, created_at);
```

Competitive Landscape

PISAMA vs Alternatives

Feature	PISAMA	LangSmith	Arize	Braintrust
Multi-agent focus	Yes	Limited	No	No
Self-healing fixes	Yes	No	No	No
Loop detection	Yes	No	No	No
Coordination analysis	Yes	No	No	No
n8n integration	Yes	No	No	No
MCP server	Yes	No	No	No

Our Moat: Purpose-built for multi-agent systems

Roadmap

What's Next

Q1 2025

- LangChain integration
- OpenAI Assistants integration
- Enhanced hallucination detection

Q2 2025

- AWS Bedrock Agents integration
- Dify / Flowise integrations
- Auto-fix application (with approval)

Pricing

Simple, Usage-Based

Tier	Price	Includes
Free	$0	1K traces/mo, basic detections
Pro	$29/mo	50K traces, all detectors, fixes
Team	$99/mo	500K traces, team dashboard, API
Enterprise	Custom	Unlimited, SLA, dedicated support

All tiers include

- All 26 detection algorithms
- Dashboard access

Getting Started

5 Minutes to First Detection

```
# 1. Install SDK
pip install mao-testing

# 2. Configure
mao config init

# 3. Add to your agent
from mao_testing import MAOTracer
tracer = MAOTracer()

with tracer.session("my-workflow") as s:
    result = my_agent.run()

# 4. View results
```

Live Demo

See PISAMA in Action

pisama.ai/demo

- Select a failure scenario
- Watch agents execute in real-time
- See detection alerts fire
- Explore fix suggestions

Or try with your own traces

1. Go to pisama.ai/diagnose
2. Paste your LangSmith/OTEL trace

The Team

Built by AI Agent Developers

- Deep experience building multi-agent systems
- Felt the pain of debugging agent failures firsthand
- Built PISAMA to solve our own problems

Why We're Different

- **Not observability retrofitted for AI** - purpose-built for agents
- **Not just logging** - intelligent detection and self-healing
- **Not framework-locked** - works with any agent system

Contact

Let's Talk

Website: pisama.ai
Demo: pisama.ai/demo
Docs: pisama.ai/docs

Ready to stop debugging agent failures manually?

Start your free trial today.

Appendix: Detection Types

#	Type	Description
F1	Specification Mismatch	Output doesn't match requirements
F2	Poor Decomposition	Subtasks not properly structured
F3	Infinite Loop	Repeated action patterns
F4	Tool Provision	Missing or inadequate tools
F5	Flawed Workflow	Graph structure issues
F6	Task Derailment	Off-topic deviation
F7	Context Neglect	Missing context usage

Appendix: Detection Types (cont.)

#	Type	Description
F8	Information Withholding	Incomplete data sharing
F9	Coordination Failure	Multi-agent sync issues
F10	Communication Breakdown	Message flow problems
F11	State Corruption	Invalid data states
F12	Persona Drift	Role inconsistency
F13	Quality Gate Bypass	Validation evasion
F14	Completion Misjudgment	Premature termination